

Week 11

Random Forests & Ensemble Methods

When One Classifier Isn't Enough

Section	Question it answers
1. Decision Trees	How does a decision tree classify, and why is a single tree unreliable?
2. From One Tree to a Forest	Why does combining unreliable trees produce a reliable classifier?
3. Testing RF on Our Data	How does a random forest compare with the six methods from Weeks 5-10?
4. Feature Importance	Which neurons and muscles drive the classification?
5. Gradient Boosting	What if each tree corrects the previous one's mistakes?
6. Bringing It Together	Where does RF fit in the complete comparison?

1. Decision Trees — The Simplest Nonparametric Classifier

How does a decision tree make predictions, and why is a single tree unreliable?

A rehabilitation engineer has now tested six classifiers on the same BCI dataset (Weeks 5–10). Each has its own failure mode: logistic regression cannot capture nonlinear interactions between muscles, Naive Bayes double-counts correlated neurons, KNN chokes on 80 dimensions, SVM works beautifully but is a black box (“which neurons are driving the decision?”), and LDA assumes every reach direction has the same neural covariance. The comparison table shows no single method wins all four tasks. The engineer asks: instead of picking one, can we combine many simple classifiers — each imperfect — into something better than any individual?

Before we can build that combination, we need to understand the building block: the decision tree. A neurologist diagnosing motor impairment already follows an implicit decision tree: is grip force below 15 N? If yes, check biceps EMG amplitude. If that is below 8 Hz, classify as impaired. A decision tree formalises this clinical triage as a learned algorithm, where the split features and thresholds are optimised from data rather than chosen by expert intuition.

The tree grows by recursive partitioning. At each node, the algorithm tries every feature and every possible threshold, looking for the split that best separates healthy from impaired trials. For example, it might ask: “is triceps activation above 11 Hz?” If the trials that answer “yes” are mostly impaired and the trials that answer “no” are mostly healthy, that is a good split.

To measure “how mixed” a group of trials is, the algorithm uses Gini impurity. Think of it as a purity score. If a node contains 60 healthy and 0 impaired trials, it is perfectly pure (Gini = 0) — no further splitting needed. If a node contains 30 healthy and 30 impaired, it is maximally mixed (Gini = 0.5 for two classes) — the split has not helped at all. The algorithm picks whichever feature and threshold produce the biggest drop in Gini impurity from parent to children. It then repeats at each child node, growing the tree one question at a time until the groups become small enough to classify confidently.

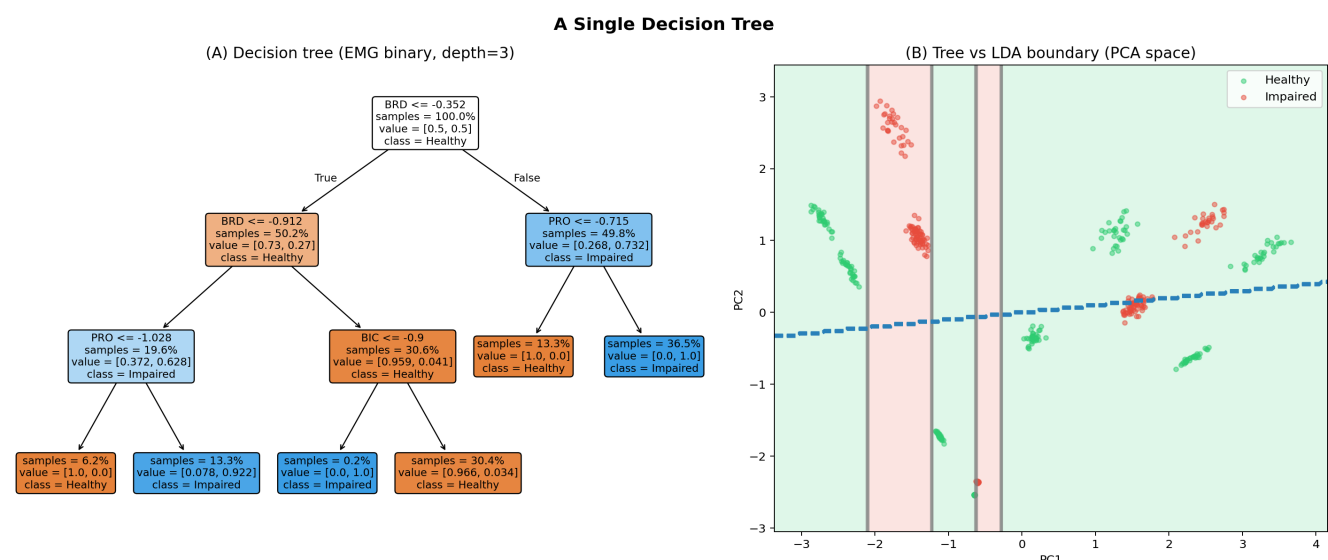


Figure 1. A single decision tree for healthy-vs-impaired classification (muscle features). (A) Tree structure (depth 4) with split features and class proportions at each leaf. (B) The resulting decision boundary in PCA space: the shaded regions show the tree's predicted class (green = healthy, red = impaired), forming piecewise-constant rectangles.

The dashed blue line is LDA's smooth linear boundary from Week 10 for comparison. The tree can capture

nonlinear regions but its edges are axis-aligned and jagged.

The danger is overfitting. Imagine a clinician who memorises every patient she has ever seen: “Patient 37 had biceps at 9.2 Hz and triceps at 11.4 Hz — impaired.” She would score 100% on those patients, but when a new patient walks in with biceps at 9.3 Hz, she has no idea what to do because she learned individual cases rather than the underlying pattern. A deep decision tree does exactly this — it keeps splitting until every training trial sits in its own leaf, achieving perfect training accuracy but failing on new subjects because it has memorised noise rather than signal.

Figure 2 shows this tradeoff — and reveals that the severity depends on the problem. Panel A uses the healthy-vs-impaired task: 6 muscle features, 2 classes, 240 trials per class. Here the tree never overfits. Training accuracy reaches 100% by depth 5, but test accuracy climbs right alongside it and stays at 99%. With only 6 features and a clean two-class split, there is simply not enough room for the tree to memorise noise — every split it finds reflects a genuine difference between healthy and impaired.

Panel B tells a different story. The neural 8-direction task has 80 features, 8 classes, and only 60 trials per class. Now the tree has 80 noisy dimensions to exploit and far fewer trials to learn from. Training accuracy still reaches 100% (by depth 8), but test accuracy peaks around depth 7 at roughly 80% and then slips. The growing orange gap between the red and green lines is the overfitting gap: the tree is fitting quirks of individual subjects’ neural firing patterns rather than the shared directional tuning that generalises across people. This is the same lesson as QDA in Week 10: when the number of parameters grows faster than the number of training examples, the model starts memorising rather than learning.

When Does a Deeper Tree Hurt?

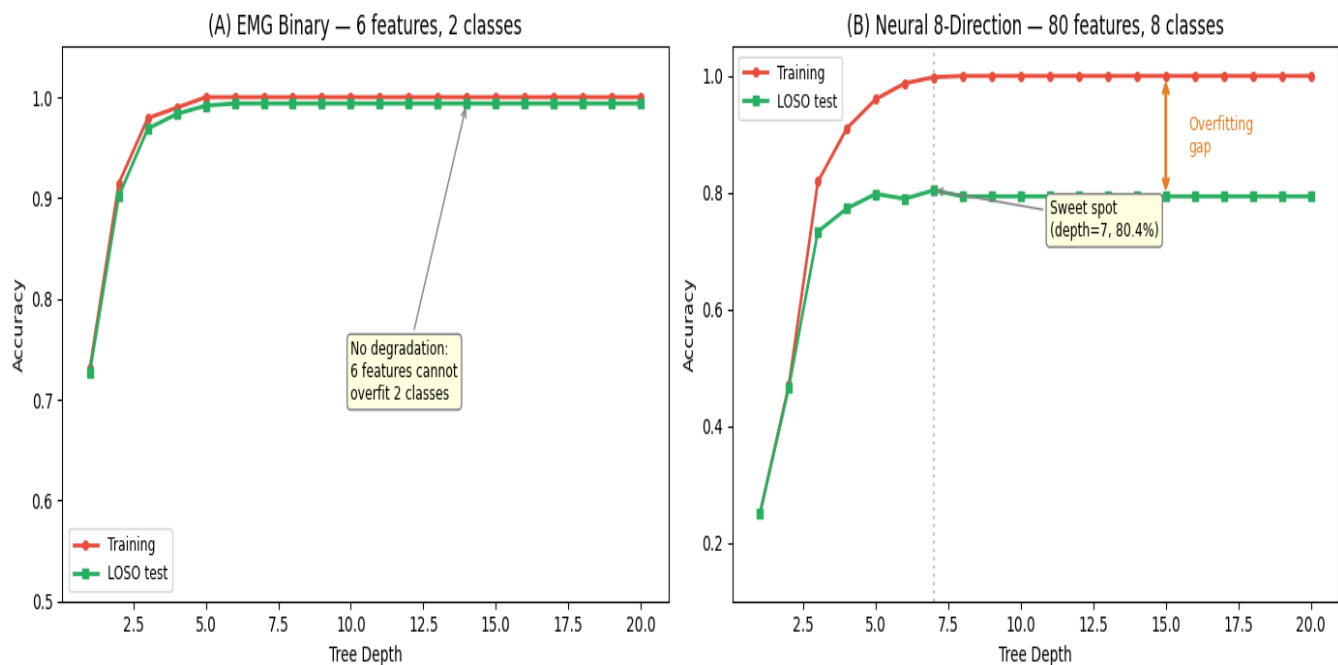


Figure 2. When does a deeper tree hurt? (A) Healthy vs impaired (6 muscle features, 2 classes): the tree never overfits because the problem is simple enough that every split the tree finds is genuine. (B) Neural 8-direction (80 features, 8 classes): training accuracy reaches 100% but test accuracy peaks and then degrades. The orange arrow marks the overfitting gap — the tree is memorising subject-specific noise in the 80 neural dimensions.

So a single decision tree is easy to understand and fast to train, but it is unreliable — especially on high-dimensional neural data. Could we fix this not by building one perfect tree, but by building many imperfect trees and letting them vote? That is the idea behind the random forest.

2. From One Tree to a Forest — The Ensemble Idea

Why does combining unreliable trees produce a reliable classifier?

Section 1 showed that a single tree overfits on hard problems: it memorises training data and fails on new subjects. The fix is surprisingly simple. Instead of building one deep tree and hoping it generalises, build 200 shallow trees — each one trained on a slightly different random subset of the data — and let them vote. Each individual tree is noisy and imperfect, but if their errors point in different directions, the majority vote will be correct far more often than any single tree. This is the wisdom of crowds, applied to classifiers.

Random forests use two mechanisms to ensure the trees' errors are independent. First, bagging (bootstrap aggregating): each tree is trained on a random bootstrap sample of the 480 trials. A bootstrap sample draws 480 trials with replacement, so each tree sees roughly 63% of the data; the remaining 37% is "out-of-bag" (OOB) and can be used for free validation. Second, random feature subsets: at each split, the tree only considers $\sqrt{p}$ features ($\sqrt{80} \approx 9$ for neural, $\sqrt{6} \approx 2$ for EMG). This decorrelates the trees — critical because if all trees split on the same dominant neuron, their errors would be correlated and averaging would not help.

Why does this work? Return to Figure 2B: a single deep tree on the neural data achieves 100% on training trials but only ~80% on held-out subjects. The tree is flexible enough to learn the correct boundaries (it gets the training data right) but it also learns spurious patterns specific to individual subjects (which is why it fails on new ones). The problem is not that the tree is wrong on average — it is that each tree is wrong in a different, unpredictable way depending on which trials it happened to see.

Averaging fixes this. Suppose a patient attempts a 90° reach. Tree 1, which happens to be confused about 45° vs 90°, incorrectly votes 45°. But trees 2 through 200 are not confused about this particular pair — they each learned from different bootstrap samples and split on different neurons — so most of them correctly vote 90°. The majority vote is 90°, and tree 1's mistake is outvoted. The same logic applies in reverse: when tree 2 makes a different mistake on a different trial, trees 1 and 3-200 outvote it. Each tree is wrong somewhere, but as long as different trees are wrong on different trials, the majority is almost always right.

The mathematics behind this is straightforward: if each tree's error is independent, the variance of the average is the variance of a single tree divided by N . With 200 trees, the forest's variability is 200 times smaller than any individual tree's. The forest keeps the tree's flexibility (it can still learn complex boundaries) but eliminates the instability (it no longer depends on which particular trials happened to end up in the training set).

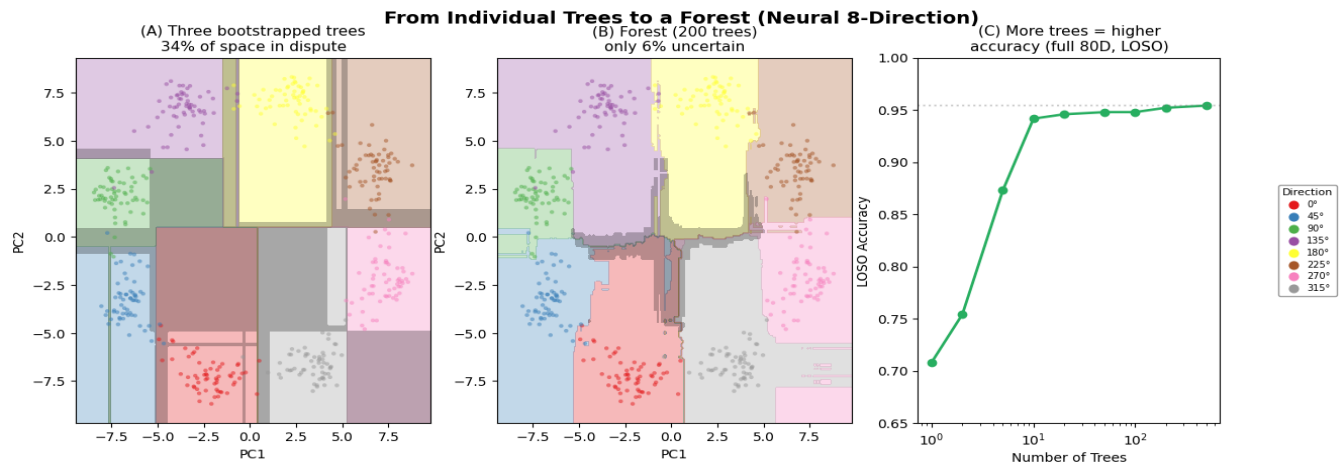


Figure 3. From individual trees to a forest (neural 8-direction in PCA space). Each colour represents one of the eight reach directions (see legend). (A) Three trees, each trained on a different bootstrap sample. The coloured regions show one tree's predictions; the darkened patches (34% of the space) mark where at least one tree disagrees. (B) The 200-tree forest's majority vote: stable predictions across most of the space, with only 6% uncertain (narrow dark bands at class boundaries). (C) LOSO accuracy on the full 80D neural data vs number of trees: accuracy climbs steeply and plateaus around 100–200 trees. Unlike tree depth (Figure 2B), adding more trees never causes overfitting.

The out-of-bag (OOB) score provides a free cross-validation estimate. Because each tree is trained on a bootstrap sample, roughly 37% of trials are left out of each tree. The forest can predict each trial using only the trees that did not see it during training, giving an unbiased accuracy estimate without the computational cost of full LOSO. Figure 4 compares OOB and LOSO estimates across our four tasks — they agree closely, confirming that OOB is a reliable proxy.

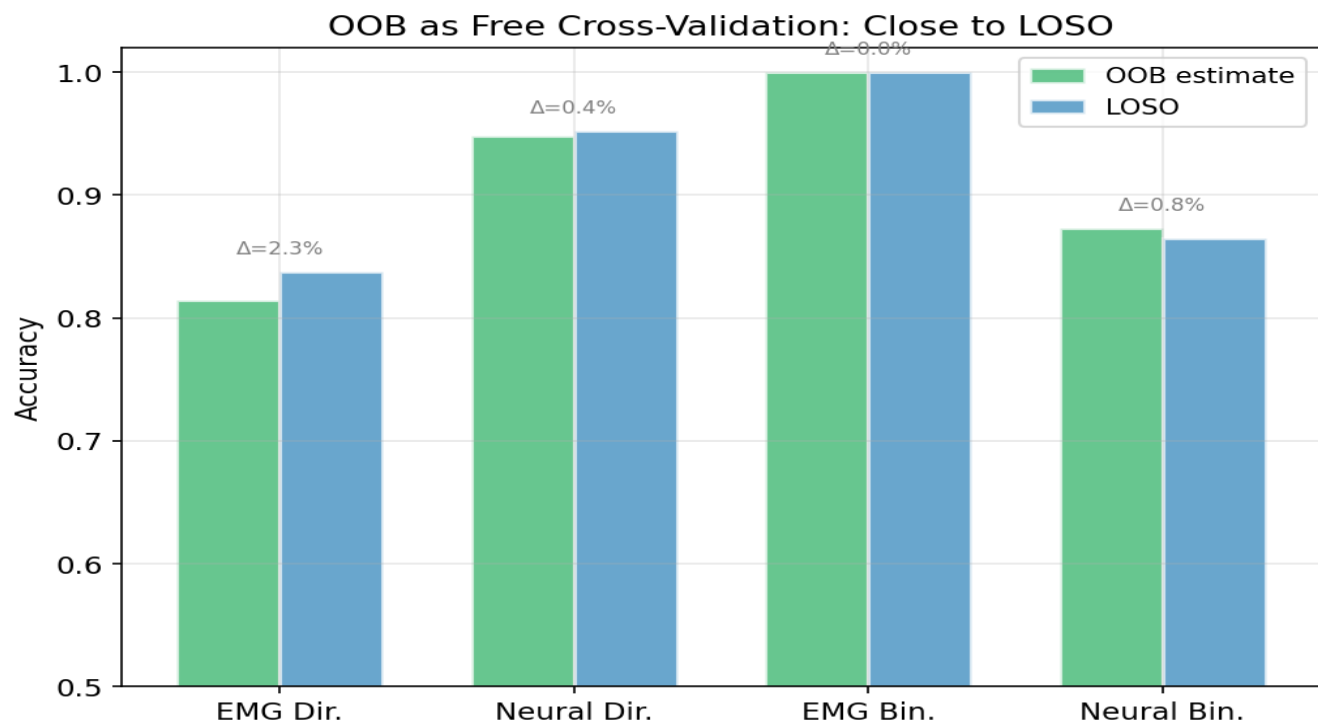


Figure 4. OOB accuracy (green) closely tracks LOSO accuracy (blue) across all four tasks. The difference (Δ) is small in every case. Because each tree is trained on a bootstrap sample, roughly 37% of trials are left out of each tree automatically. The forest predicts each trial using only the trees that never saw it — no additional models need to be trained. This makes OOB a built-in accuracy estimate that comes at no extra computational cost, unlike

LOSO which requires re-training 20 separate forests.

The random forest algorithm, step by step

To make the ensemble idea concrete, here is exactly what happens when we train a 200-tree random forest on the neural 8-direction task (480 trials from 20 subjects, 80 neurons):

Training — repeat 200 times, once per tree:

- Step 1 — Bootstrap. Draw 480 trials with replacement from the training set. Some trials appear twice, some not at all. Roughly 63% of the original trials end up in this sample; the other 37% are this tree's out-of-bag trials.
- Step 2 — Grow a tree. Starting from the root, pick 9 neurons at random ($\sqrt{80} \approx 9$). For each of these 9 neurons, find the firing-rate threshold that best separates the 8 directions (lowest Gini impurity). Split on whichever neuron-threshold pair wins.
- Step 3 — Repeat at each child. At every new node, pick a fresh random set of 9 neurons and find the best split among those. Continue until each leaf is pure (all trials from one direction) or contains very few trials. Because each node sees a different random subset of neurons, the tree ends up relying on a different combination of neurons than any other tree in the forest.

Prediction — for a new trial:

- Step 4 — Drop the trial down all 200 trees. Each tree asks its sequence of questions ("is neuron 34 above 12.1 Hz? is neuron 7 above 8.3 Hz?") and arrives at a leaf. During training, that leaf collected trials from one or more directions; the leaf's vote is whichever direction was most common among its training trials.
- Step 5 — Majority vote. Each tree casts the vote it learned from the labelled training data. Count the votes: if 112 trees say 90°, 43 say 135°, and the rest scatter across other directions, the forest predicts 90°. The margin (112 vs 43) also gives a rough confidence estimate.

This is supervised classification — the trees can only vote for directions they saw during training.

The key insight is that no single tree sees all 80 neurons at every split. Tree 1 might rely heavily on neurons 7, 34, and 52; tree 2 might rely on neurons 11, 28, and 63. If neuron 34 happens to be noisy for a particular subject, tree 1 gets confused but tree 2 is unaffected. Averaging across 200 such trees cancels out the individual errors, which is why the forest is more stable than any single tree.

Notice that several choices in the algorithm above are not fixed by the data — they are hyperparameters that the user sets before training. The four most important are: (1) `n_estimators` — how many trees to grow (we chose 200 in Step 1); (2) `max_depth` — how deep each tree can grow, i.e. how many questions it can ask (we left this unlimited in Step 3, letting trees grow until leaves are pure); (3) `max_features` — how many neurons each split considers (the 9 in Step 2, set to $\sqrt{p}$ by default); and (4) `min_samples_leaf` — the smallest number of trials allowed in a leaf before the tree stops splitting (we used 1). Unlike SVM's C and γ which required careful grid search in Week 9, these settings rarely need tuning — we will confirm this in Section 3.

Hyperparameters are what you choose before training. The parameters are what the forest learns from the data. For each of the 200 trees, the learned parameters are: which neuron to split on at

every node, the firing-rate threshold for that split, and the predicted direction at each leaf. On our dataset, each tree grows to an average depth of about 9 with roughly 100 nodes, each storing a neuron index and a threshold — so the full 200-tree forest learns on the order of 40,000 parameters. Compare this with LDA, which learns a single projection matrix ($80 \times 7 = 560$ numbers), or logistic regression, which learns 80 coefficients. The forest's larger parameter count is what gives it flexibility; bagging and random feature subsets are what prevent that flexibility from becoming overfitting.

3. Testing Random Forests on Our Data

How does a random forest compare with the six methods from Weeks 5–10?

Section 2 described how a random forest works in principle. Now we test it on our data. We train a 200-tree forest with the default settings from the algorithm walkthrough ($\sqrt{p}$ features per split, unlimited depth, 1 trial per leaf) and evaluate it on all four tasks using the same leave-one-subject-out protocol from previous weeks. Figure 5 adds RF (highlighted with green edges) to the six-method comparison from Week 10.

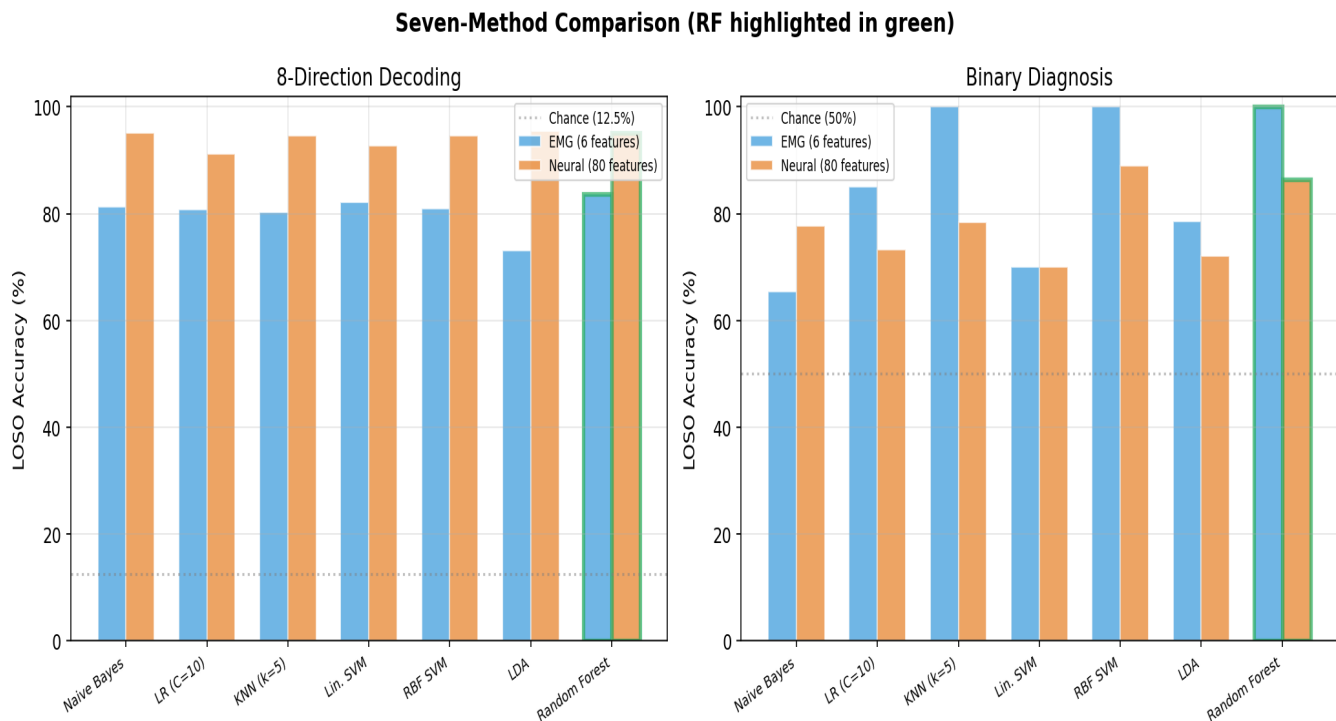


Figure 5. Seven-method LOSO comparison across four tasks (extending Week 10 Figure 5 with RF added, green edges). Left: 8-direction decoding (chance = 12.5%). Right: binary diagnosis (chance = 50%). RF is competitive with the best method on every task without any task-specific tuning.

RF is competitive across all four tasks. On neural 8-direction decoding, RF matches Naive Bayes and LDA at the top. On EMG direction decoding, RF edges ahead of all previous methods. On the binary tasks, RF achieves perfect or near-perfect accuracy. Unlike SVM (which needed a $C \times \gamma$ grid search in Week 9) or KNN (which needed k selection in Week 8), RF works well straight out of the box with default settings.

But how sensitive is RF to those default settings? Section 2 introduced the four hyperparameters and claimed they rarely need tuning. Figure 6 tests this claim directly. Each panel varies one hyperparameter across a wide range while holding the other three at their defaults. The y-axis in every panel is 5-fold cross-validation accuracy on the neural 8-direction task — higher means more trials are classified correctly.

Panel (A) varies the number of trees from 1 to 500. A single tree scores only about 75%, but accuracy climbs rapidly — by 10 trees it reaches 93%, and it plateaus around 100 trees. Going from 100 to 500 adds computation but not accuracy. Panel (B) varies maximum tree depth. Shallow trees (depth 2) underfit because they cannot ask enough questions to separate 8

directions, but once depth exceeds about 7, accuracy levels off. Unlimited depth (the default) is fine because the forest's averaging prevents the individual-tree overfitting we saw in Figure 2. Panel (C) varies how many neurons each split considers. The default $\sqrt{p} \approx 9$ sits near the peak, but the curve is broad: anywhere from 3 to 30 features gives similar results. Panel (D) varies the minimum leaf size. Requiring larger leaves (10+) forces the trees to stop early, reducing their ability to separate closely spaced directions. The default of 1 works well.

The overall message: unlike SVM's sharp sensitivity to C and γ (Week 9, Figure 3), RF's accuracy is flat across a wide range for every hyperparameter. The defaults are safe, and a practitioner can trust them without running an expensive grid search.

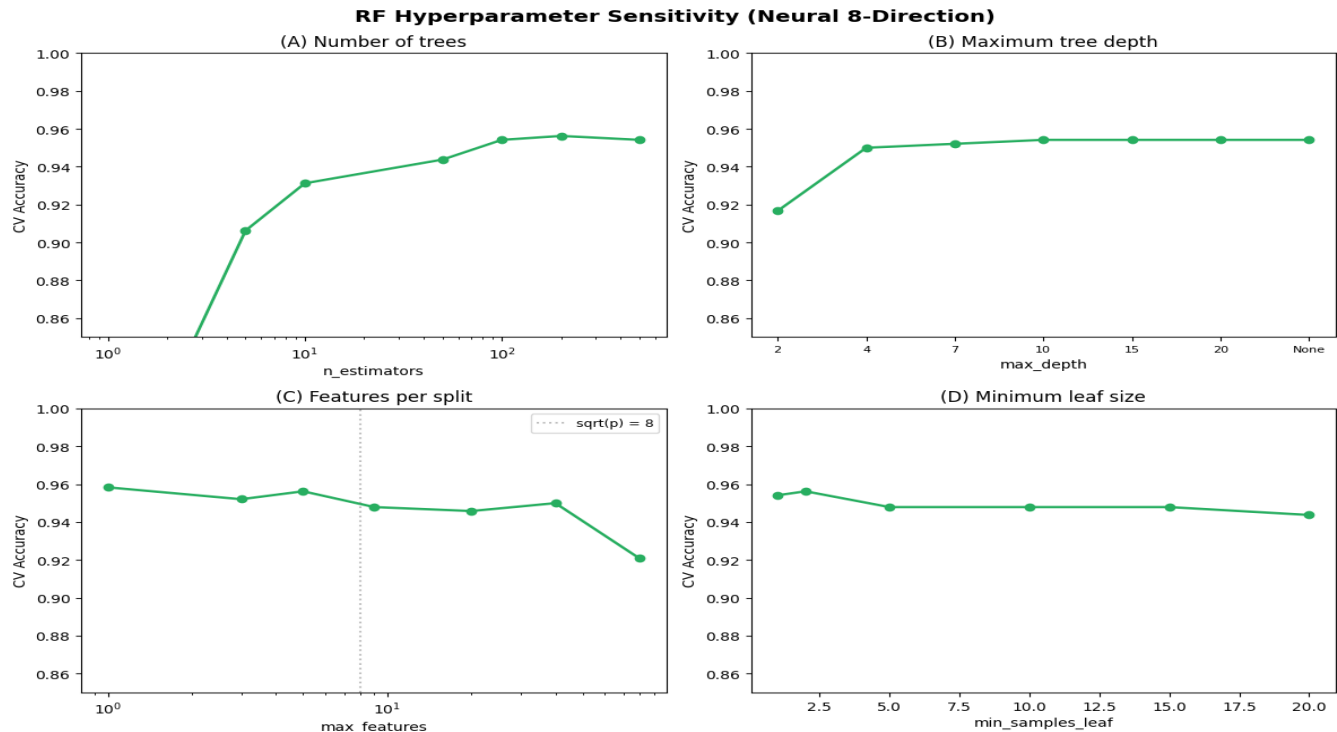


Figure 6. RF hyperparameter sensitivity (neural 8-direction). Each panel varies one hyperparameter while holding the others at their defaults. The y-axis is 5-fold cross-validation accuracy (proportion of trials classified correctly). (A) Number of trees: plateaus by ~ 100 . (B) Maximum depth: no benefit beyond 7. (C) Features per split: broad peak around the default $\sqrt{p} \approx 9$. (D) Minimum leaf size: degrades only above 10. Default values (200 trees, unlimited depth, $\sqrt{p}$ features, 1 sample per leaf) work well without tuning.

4. Feature Importance — What the Forest Learned

Which neurons and muscles drive the classification, and does the forest agree with PCA and LDA?

Sections 2–3 showed that RF classifies accurately. But in a clinical setting, accuracy alone is not enough. If a rehabilitation engineer tells a neurologist “our decoder is 95% accurate,” the neurologist’s next question is: “which neurons are doing the work?” If specific neurons drive the classification, those neurons might correspond to a recoverable cortical subregion, or they might reveal which aspect of the movement is most affected by impairment. SVM and KNN gave us accuracy in Weeks 8–9 but could not answer this question — they are black boxes. RF can.

The tool is permutation importance. The idea is simple: after training the forest, take one feature — say, neuron 34 — and randomly shuffle its values across trials, breaking its relationship with the reach direction. Then re-measure the forest’s accuracy. If accuracy drops sharply, neuron 34 was important; if accuracy barely changes, the forest did not rely on it. Repeat for all 80 neurons (or all 6 muscles) to get a complete ranking. This is more reliable than the alternative (impurity-based importance) because it measures actual prediction impact rather than how often a feature was used for splitting.

Figure 7 compares four methods’ answers to the question “which muscles matter for diagnosis?” on the healthy-vs-impaired classification using muscle recordings. Panel A shows RF permutation importance; Panel B shows LDA’s Fisher weights from Week 10; Panel C shows logistic regression coefficients from Week 5; Panel D shows PCA loadings from Week 4.

The four methods do not fully agree, and the disagreement is instructive. RF (A) puts almost all importance on the brachioradialis (BRD), with every other muscle near zero. LDA (B) and LR (C) emphasise the posterior deltoid (PD) and the biceps/triceps pair (BIC, TRI), with BRD only mid-range. PCA (D) highlights TRI and PRO as the largest contributors to total variance. Why the discrepancy? Each method asks a different question. PCA asks “which muscles vary the most?” — that can include variation unrelated to impairment. LDA and LR ask “which muscles separate the two classes linearly?” RF asks “which muscles, when scrambled, hurt prediction the most?” If BRD carries a nonlinear or interaction-based signal that LR’s linear coefficients miss, RF will detect it while LR will not. The lesson: feature importance rankings depend on the method, and comparing multiple methods reveals richer structure than any single ranking.

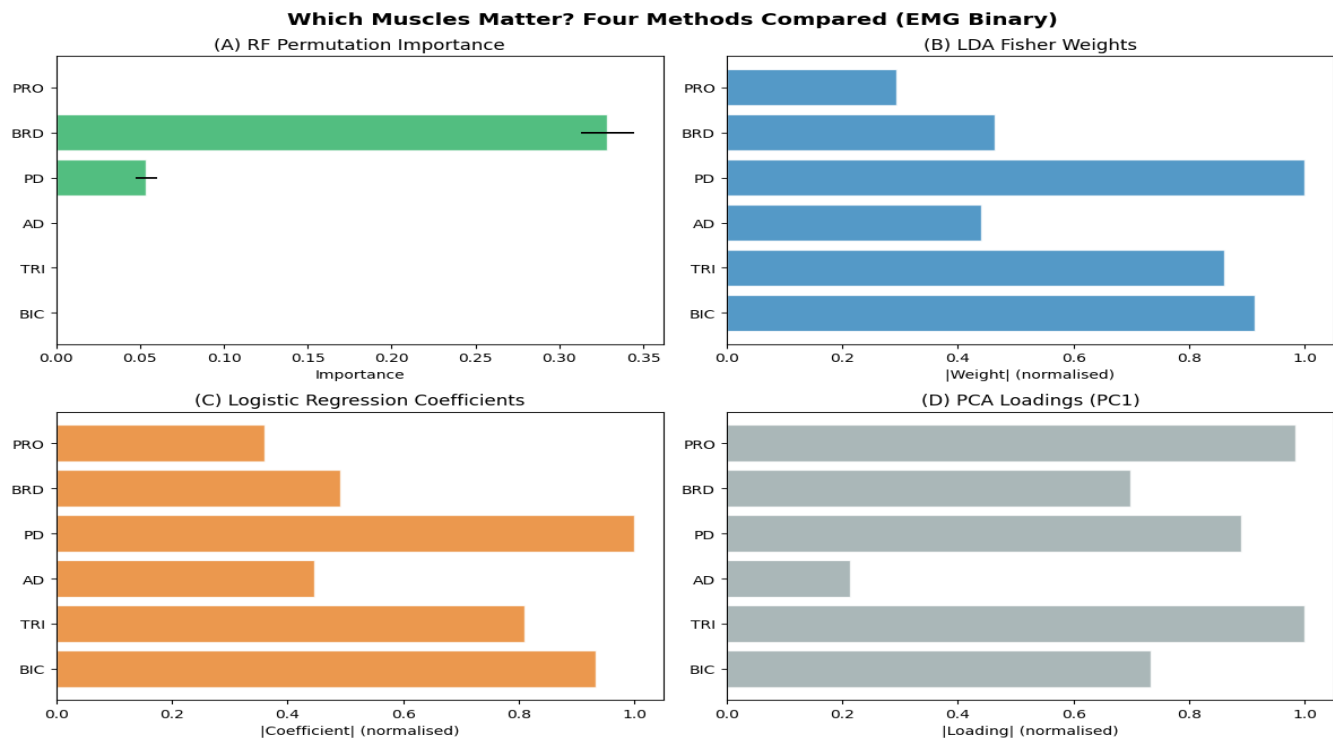


Figure 7. Which muscles matter for diagnosis? Four methods compared (healthy vs impaired, muscle features). (A) RF permutation importance: each muscle's values are shuffled 10 times, producing a mean and standard deviation (error bars). RF places almost all importance on BRD. (B) LDA Fisher weights (Week 10): PD and BIC/TRI dominate. (C) Logistic regression coefficients (Week 5): similar to LDA. (D) PCA loadings on PC1 (Week 4): TRI and PRO lead, but PCA captures total variance, not class separation. Panels B–D have no error bars because they compute single deterministic values. The disagreement across methods reflects different definitions of “importance.”

The clinical value of feature importance is not that it gives a single definitive answer, but that it generates hypotheses a clinician can investigate. If RF highlights BRD while LDA highlights PD, the clinician now has two leads: perhaps BRD carries a nonlinear impairment signature (detectable by RF but invisible to linear methods), while PD shows the clearest linear separation between groups. Both are worth investigating. By contrast, SVM and KNN from Weeks 8–9 classify accurately but offer no starting point for this kind of clinical reasoning — they are black boxes that say “impaired” without explaining why.

Figure 8 applies this to the neural 8-direction task. Each bar shows one neuron's permutation importance, computed on a held-out test set (30% of trials) to avoid the memorisation problem from Figure 2 — on the training data, the forest's perfect accuracy would make every neuron appear unimportant. The bars are coloured by each neuron's preferred direction from the cosine tuning model in Week 6, and the error bars show variability across 10 shuffling repetitions.

Two features stand out. First, no single neuron dominates: the most important neuron (N75) accounts for only about 0.01 of accuracy, and importance is spread across neurons with all eight preferred directions. This is consistent with the cosine tuning model from Week 6, where every neuron contributes some directional information. Second, some neurons have near-zero or slightly negative importance, meaning that scrambling them does not hurt (or even slightly helps) the forest. These are neurons whose firing rates add noise rather than signal for direction decoding. A clinician designing a BCI could consider dropping them to simplify the decoder without losing accuracy.

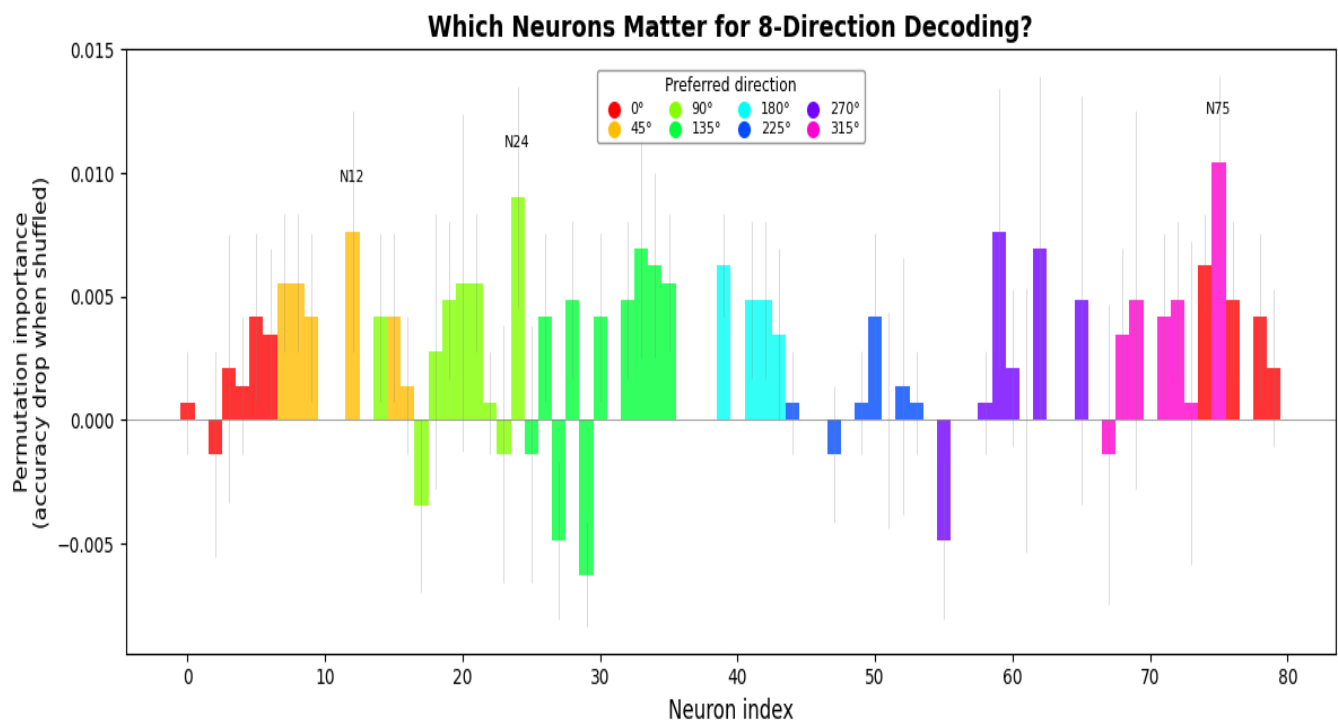


Figure 8. Permutation importance of all 80 neurons for 8-direction decoding (computed on held-out test data). Bars are coloured by preferred direction (see legend). Error bars show variability across 10 shuffling repetitions. The y-axis is the drop in accuracy when that neuron's values are scrambled: higher bars = more important neurons. Importance is distributed across the population, with no single neuron indispensable.

5. Gradient Boosting — A Different Ensemble Strategy

What if instead of averaging independent trees, we build each tree to fix the previous one's mistakes?

A random forest trains all its trees independently and takes a majority vote. Gradient boosting takes a different approach that will feel familiar to anyone who has studied motor learning: iterative error correction. A person learning to reach toward a target does not start over from scratch after each attempt. Instead, the motor system measures the error on the current reach, then adjusts the motor command to correct that specific error on the next attempt. Gradient boosting does the same thing with decision trees.

The first tree classifies all 480 trials as best it can, but inevitably gets some wrong — perhaps it confuses 45° and 90° reaches because their neural patterns overlap. Before training the second tree, the algorithm increases the weight on those misclassified trials so they count more heavily in the next round. The second tree, seeing these harder trials as more important, adjusts its splits to get them right — even if that means sacrificing accuracy on some previously-correct trials. The third tree then upweights whatever the first two still miss. After 200 rounds, the final prediction combines all the trees, each one contributing a small correction to the errors of those before it. Each individual tree is shallow (only 3–5 questions deep) and weak on its own, but the cumulative sequence of corrections is powerful — just as hundreds of small reach corrections can converge on a precise movement.

The tradeoff is straightforward: random forests reduce variance (stabilising noisy predictions by averaging), while boosting reduces bias (fixing systematic errors by targeting mistakes). On our cosine-tuned neural data, where the class boundaries are nearly linear, the forest already achieves high accuracy — there is little systematic error left for boosting to correct. Figure 9 confirms this: the two strategies achieve similar accuracy across all four tasks.

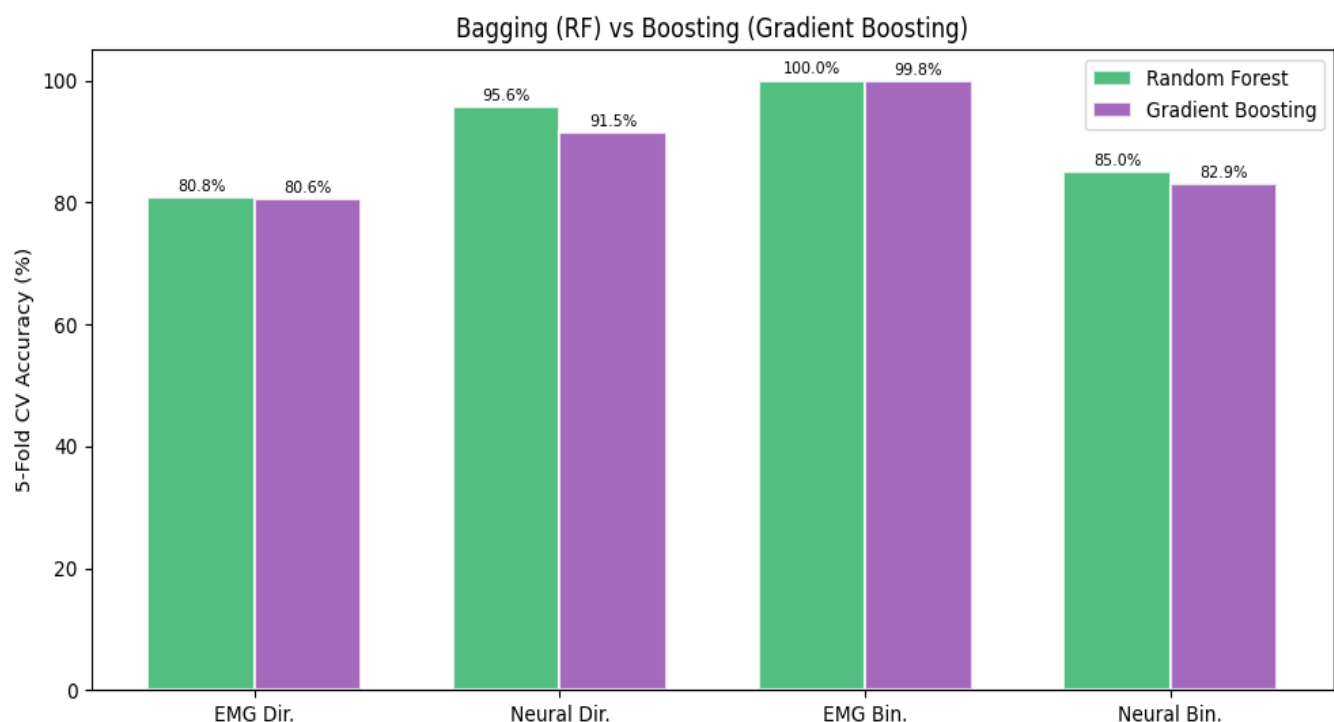


Figure 9. Random Forest vs Gradient Boosting on all four tasks (5-fold CV). Both ensemble strategies achieve similar accuracy on this dataset. Boosting would show a larger advantage on data with complex, nonlinear class boundaries that a single forest pass cannot capture.

When would boosting matter more? If the impairment in our dataset produced subtly nonlinear changes in neural coordination — for example, if impaired subjects showed altered timing between neuron pairs rather than simple amplitude changes — a random forest might miss the pattern, while boosting's iterative correction could learn it. In practice, libraries like XGBoost and LightGBM are the most common boosting implementations and are widely used in clinical prediction and biomarker studies. For this course, the key takeaway is that there are two flavours of ensemble: parallel averaging (RF) and sequential correction (boosting), and the choice depends on whether your problem is limited by variance or by bias.

6. Bringing It Together

Where does Random Forest fit in the complete comparison, and when should you prefer it?

Computational cost. RF trains in parallel (each tree is independent) and is fast to fit: roughly 455 ms for 200 trees on our 480×80 neural dataset. Prediction is slower than LDA (which is a single matrix multiply) but faster than KNN (which requires distance computations to all training points at test time). Figure 10 shows the speed-accuracy tradeoff for all seven methods.

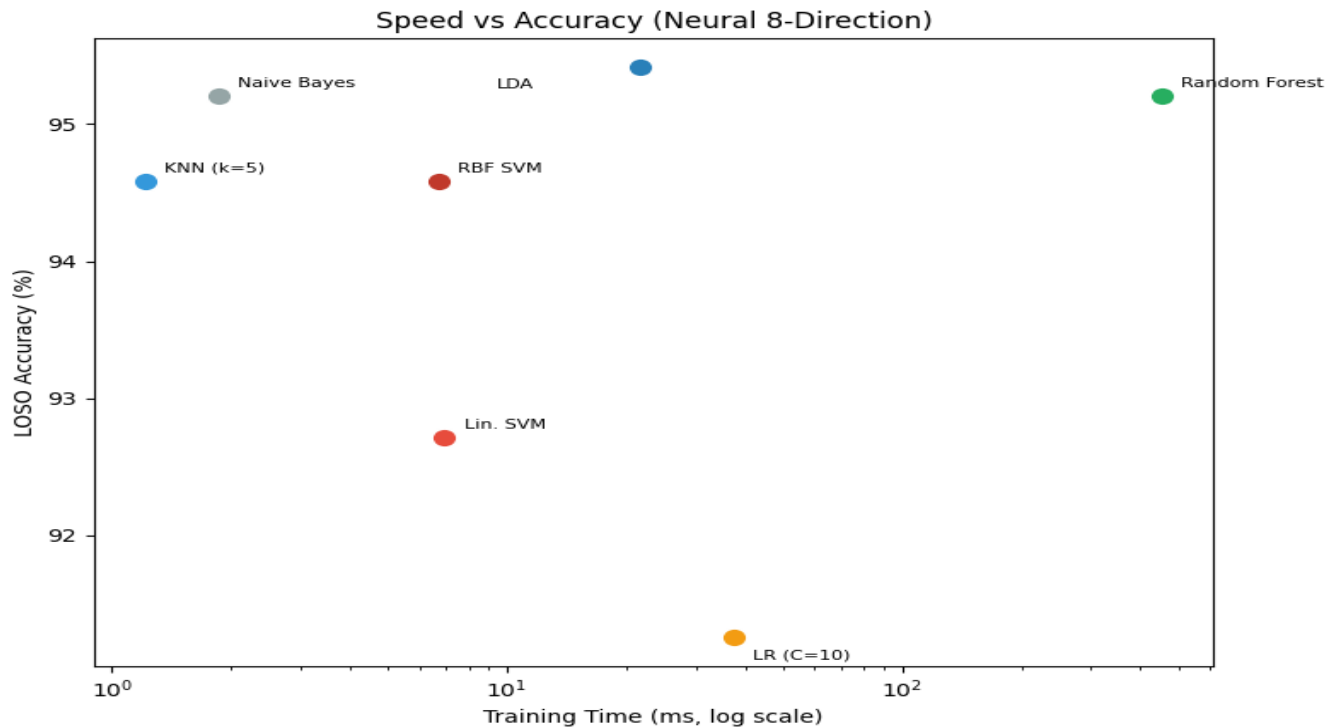


Figure 10. Training time vs LOSO accuracy (neural 8-direction). Each point is one method. RF occupies a good position: high accuracy at moderate training cost. LDA and Naive Bayes are faster but RF is more versatile across tasks.

Electrode drift robustness. We repeat the drift simulation from Weeks 9–10, adding Gaussian noise with increasing σ to the neural features. Figure 11 shows how all seven methods degrade. RF degrades at a rate comparable to the best linear methods. Because each tree has learned different feature subsets, the forest partially compensates when a few neurons drift — the trees that relied on drifted neurons become less accurate, but the trees that relied on stable neurons maintain their contribution.

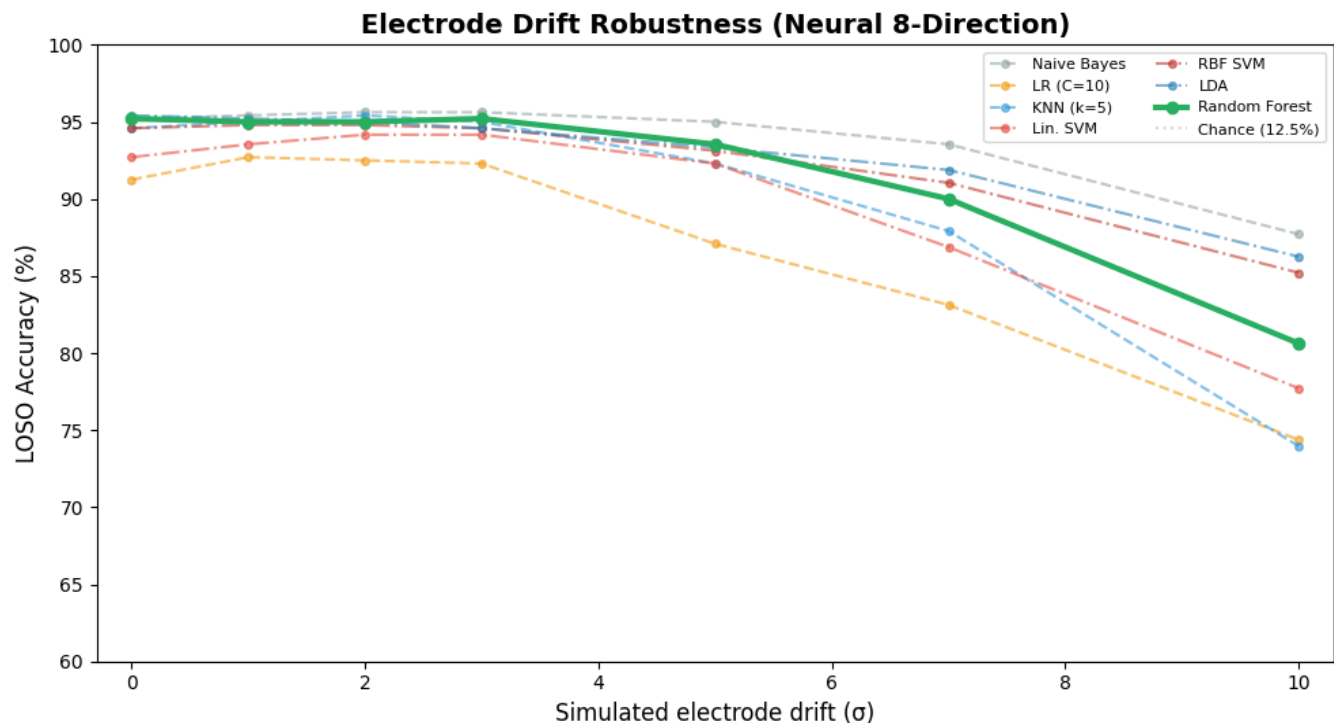


Figure 11. Accuracy under simulated electrode drift (neural 8-direction). RF (green, thick line) degrades at a rate comparable to other methods. Its distributed reliance on many feature subsets provides moderate robustness to individual neuron drift.

The updated comparison.

Summary Comparison — All Seven Classifiers (LOSO)

	NB	LR	KNN	Lin SVM	RBF SVM	LDA	RF
EMG Dir.	81.2%	80.8%	80.2%	82.1%	81.0%	73.1%	83.8%
Neural Dir.	95.2%	91.2%	94.6%	92.7%	94.6%	95.4%	95.2%
EMG Bin.	65.4%	85.0%	100.0%	70.0%	100.0%	78.5%	100.0%
Neural Bin.	77.7%	73.3%	78.3%	70.0%	89.0%	72.1%	86.5%
Train (ms)	1.9	26.9	12.3	3.5	6.7	24.4	422.4
Predict (ms)	1.13	0.28	6.49	3.96	9.96	0.33	15.54
Hyperparams	None	C	k, metric	C	C, γ , kernel	None	Few (insensitive)
Interpretability	Priors/likelihoods	Coefficients	None	Weights	None	Loadings	Feature importance

Figure 12. Summary comparison table for all seven classifiers through Week 11. RF (highlighted in green) adds a new Interpretability column: feature importance is the only method that provides built-in feature ranking while remaining competitive across all tasks. Compare with LDA (loadings, but assumes Gaussian classes) and LR (coefficients, but assumes linear boundaries).

When to use Random Forests:

- When you need a strong baseline with minimal tuning. Unlike SVM, which required a grid search over C and γ in Week 9, RF works well with default settings.
- When feature importance or biomarker discovery is a goal. RF can tell a clinician which neurons or muscles drove the classification — something KNN and SVM cannot do.

- When you do not know the decision boundary shape (linear vs nonlinear). RF makes no assumptions about how neural activity relates to movement.
- When features may interact in complex ways. For example, impairment might only be detectable when two muscles are simultaneously abnormal — RF captures this through its branching structure.

When NOT to use Random Forests:

- When you need well-calibrated probabilities. If a screening tool says “92% chance of impairment,” a clinician expects that about 92 out of 100 such patients are truly impaired. RF’s probability estimates are based on the fraction of trees that voted for each class, and in practice these votes tend to be lopsided — the forest is usually either very sure or very unsure, with little in between, making the reported percentages unreliable as true confidence scores.
- When you need a compact model for real-time BCI. Running a trial through 200 trees is slower than LDA’s single matrix multiplication.
- When the decision boundary is truly linear. Our cosine-tuned neural data is a good example — LDA achieves the same accuracy with a simpler model.

What we learned this week

- Decision trees are intuitive but overfit; forests fix this by averaging many decorrelated trees.
- RF is the first method competitive across all four tasks without tuning.
- Feature importance lets the forest explain which features drive its predictions. Different methods can disagree on what is important, revealing richer structure than any single ranking.
- Boosting is an alternative ensemble strategy that trains trees sequentially, each one correcting the previous one’s mistakes.
- The right method depends on the question: interpretability (RF), real-time speed (LDA), calibrated probabilities (LR), nonlinear boundaries (RBF SVM), and dimensionality reduction (PCA/LDA) are all legitimate goals.

References

Breiman L. Random Forests. *Machine Learning*, 45(1), 5-32, 2001.

Friedman JH. Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, 29(5), 1189-1232, 2001.

Strobl C, Boulesteix AL, Kneib T, Augustin T, Zeileis A. Conditional variable importance for random forests. *BMC Bioinformatics*, 9, 307, 2008.

Churchland MM, Cunningham JP, Kaufman MT, Foster JD, Nuyujukian P, Ryu SI, Shenoy KV. Neural population dynamics during reaching. *Nature*, 487(7405), 51-56, 2012.

Georgopoulos AP, Kalaska JF, Caminiti R, Massey JT. On the relations between the direction of two-dimensional arm movements and cell discharge in primate motor cortex. *Journal of Neuroscience*, 2(11), 1527-1537, 1982.